Smart Order Management System

Microservices Architecture

Phase 1 & Enterprise Edition

MAP (Middleware and Application Programming)

Assignment I & II — Architecture Report

Sneh Patel	Milan Poria
25MCA035	25MCA037

Institute of Technology, Nirma University | Ahmedabad — 2026

GitHub Repository: github.com/milan-sde/MicroServicesProject

1. Introduction

This document presents the architecture and design rationale for the Smart Order Management System (SOMS) — a full-stack web application developed using a Microservices Architecture. The project was completed in two phases as part of the MAP (Middleware and Application Programming) course at the Institute of Technology, Nirma University.

The system simulates a real-world e-commerce order management platform where multiple independent services collaborate over REST APIs and gRPC to handle users, products, inventory, orders, payments, and notifications. The two assignments progressively evolved the architecture from a simple Phase 1 microservices setup to a production-grade Enterprise architecture with event-driven communication, circuit breakers, and Docker containerization.

1.1 Objectives

- Implement a scalable Microservices Architecture using Node.js services.
- Use Nginx as an API Gateway for routing REST API calls from the React client.
- Implement gRPC for high-performance inter-service communication.
- Apply MongoDB with separate collections per service (database-per-service pattern).
- Integrate ActiveMQ for asynchronous event-driven messaging (Enterprise phase).
- Implement Circuit Breaker pattern for fault tolerance in critical service calls.
- Containerize all services using Docker for consistent deployment.

1.2 Technology Stack

React + Vite Frontend client UI (SPA — Single Page Application)	**Nginx** API Gateway and reverse proxy for routing
Node.js Runtime for all backend microservices	**MongoDB** NoSQL database — one collection per microservice
gRPC High-performance binary RPC protocol for inter-service calls	**ActiveMQ** Message broker for event-driven / async communication
Docker Containerization of each microservice	**JWT** JSON Web Tokens for stateless authentication
Circuit Breaker Fault tolerance pattern (Opossum library)	**REST API** HTTP-based API for client-to-gateway communication

2. Assignment I — Phase 1 Microservices Architecture

Assignment I implements a foundational microservices system with five Node.js services communicating via REST (to the Nginx Gateway) and gRPC (between the Order Service and downstream services). All services share a single MongoDB instance but maintain separate collections, following the database-per-service pattern.

2.1 Architecture Diagram

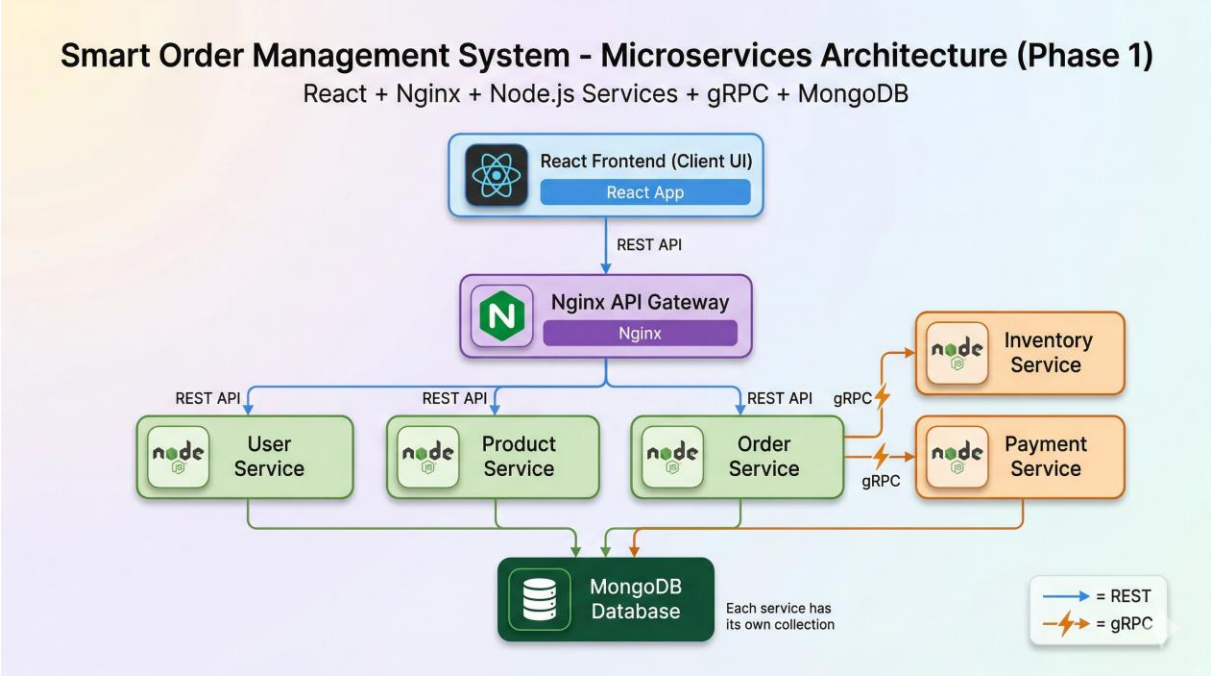

Figure 1: Smart Order Management System — Phase 1 Microservices Architecture

2.2 Architecture Layers

2.2.1 Client Layer — React Frontend

The client-facing interface is a Single Page Application (SPA) built with React. It communicates exclusively with the Nginx API Gateway over HTTP REST. The frontend handles views for product listing, cart, user authentication, and order placement.

- Framework: React (with Vite build tool)
- Communication Protocol: REST API over HTTP
- Responsibilities: UI rendering, state management, API consumption

2.2.2 API Gateway Layer — Nginx

Nginx serves as the API Gateway — the single entry point for all client requests. It performs reverse proxying and request routing to the appropriate microservice based on URL path. Nginx also provides load balancing capability and SSL termination in production.

- Technology: Nginx (configured via nginx.conf)
- Function: Reverse proxy, request routing, load balancing
- Incoming: REST from React frontend
- Outgoing: REST to User, Product, and Order services

2.2.3 Microservices Layer

Five independent Node.js services each own a specific bounded context:

1. **User Service** — Handles user registration, login, and profile management. Stores data in the Users collection of MongoDB.
2. **Product Service** — Manages the product catalog including CRUD operations for product listings. Stores in the Products collection.
3. **Order Service** — Core orchestration service. Places and tracks orders. Communicates with Inventory and Payment services via gRPC.
4. **Inventory Service** — Tracks stock levels for products. Receives gRPC calls from the Order Service to check/reserve inventory.
5. **Payment Service** — Processes payment transactions triggered by the Order Service via gRPC.

2.2.4 Communication Protocols

Two distinct communication protocols are used based on the nature of the interaction:

- REST API (HTTP): Used between the React client and Nginx, and between Nginx and the three primary services (User, Product, Order). REST is chosen for its simplicity and compatibility with browser clients.
- gRPC: Used between the Order Service and the Inventory and Payment services. gRPC uses Protocol Buffers (protobuf) for serialization, offering significantly lower latency and higher throughput than REST — ideal for backend-to-backend synchronous calls.

2.2.5 Data Layer — MongoDB

A single MongoDB instance hosts separate collections for each service: users, products, orders, inventory, and payments. Each service accesses only its own collection — enforcing the database-per-service principle, ensuring loose coupling and independent scalability.

3. Assignment II — Enterprise Microservices Architecture

Assignment II builds upon the Phase 1 foundation and evolves the system into a production-grade Enterprise Microservices Architecture. The key additions are: Docker containerization, ActiveMQ message broker for async communication, Circuit Breaker protection on critical gRPC paths, and a new Notification Service as an event consumer.

3.1 Architecture Diagram

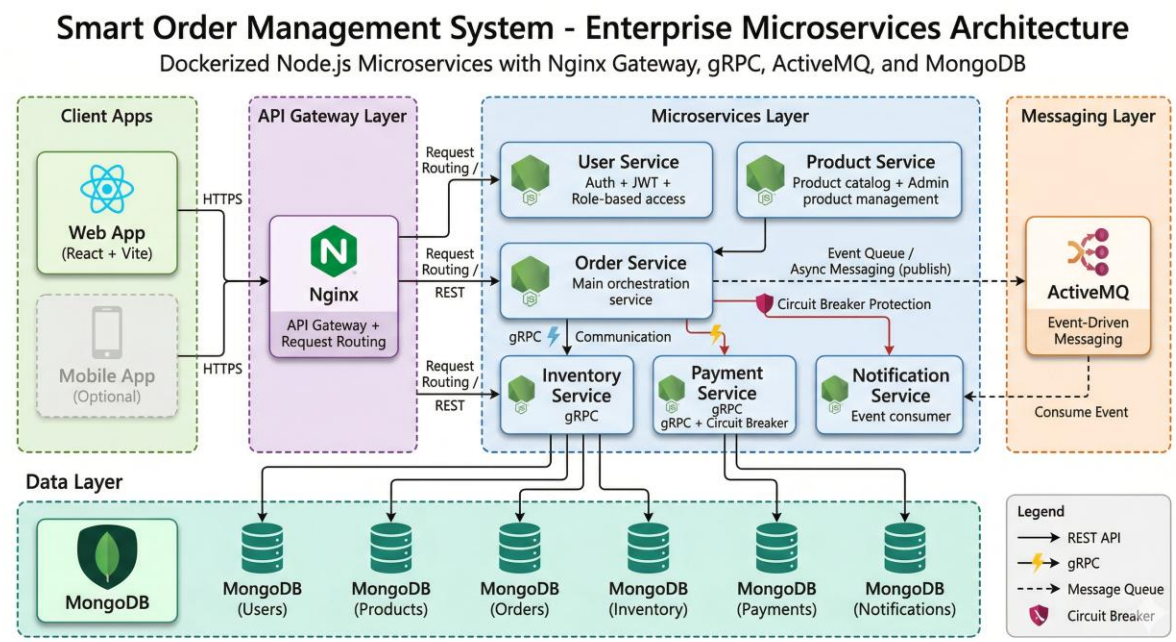

Figure 2: Smart Order Management System — Enterprise Microservices Architecture

3.2 Enterprise Architecture Enhancements

3.2.1 Dockerized Deployment

All microservices, Nginx, MongoDB, and ActiveMQ are containerized using Docker. A docker-compose.yml file orchestrates the entire system, defining service dependencies, network bridges, and environment variables. This ensures environment consistency across development and production.

- Each service has its own Dockerfile.
- Docker Compose manages inter-container networking.
- Named volumes persist MongoDB data across container restarts.

3.2.2 Expanded Microservices

The Enterprise edition adds a sixth service — the Notification Service:

- User Service: Auth + JWT-based authentication + Role-based access control (RBAC). Users can have roles like admin or customer, and protected routes are guarded via JWT middleware.
- Product Service: Full product catalog management including Admin-only product creation and updates.
- Order Service: Main orchestration service — receives order requests, coordinates with Inventory and Payment via gRPC, and publishes events to ActiveMQ upon successful order placement.

- Inventory Service: gRPC server handling stock queries and reservations from the Order Service.
- Payment Service: gRPC server with Circuit Breaker protection. Handles payment processing and is protected from cascading failures.
- Notification Service: An ActiveMQ event consumer. Listens for 'order.placed' events published by the Order Service and triggers notifications (e.g., email or SMS alerts).

3.2.3 ActiveMQ — Event-Driven Messaging

ActiveMQ acts as a Message Broker implementing the publish-subscribe and point-to-point messaging patterns. The Order Service publishes an event to a topic/queue in ActiveMQ after a successful order. The Notification Service subscribes to that queue and processes events asynchronously — decoupling the two services and ensuring the Order Service is never blocked by notification logic.

- Protocol: STOMP or AMQP over ActiveMQ
- Publisher: Order Service (publishes 'order.placed' event)
- Consumer: Notification Service (subscribes to 'order.placed' queue)
- Pattern: Event-Driven Architecture (EDA) using Message Queues

3.2.4 Circuit Breaker Pattern

The Circuit Breaker pattern is implemented on the Order Service to Payment Service gRPC call. If the Payment Service becomes unavailable or repeatedly fails, the Circuit Breaker opens — preventing further calls and returning a fallback response immediately. This avoids cascading failures across the system.

- Library: Opossum (Node.js Circuit Breaker library)
- States: Closed (normal), Open (failing — no calls), Half-Open (testing recovery)
- Protected call: Order Service to Payment Service (gRPC)

4. Key Concepts and Keywords

The following table summarizes all major architectural and technical concepts used across both assignments:

Microservices Architecture Design pattern where an application is built as a suite of small, independently deployable services, each with its own business responsibility.	**API Gateway** A single entry point that handles all client requests and routes them to the appropriate microservice — implemented with Nginx in this project.
gRPC (Google Remote Procedure Call) A high-performance, language-agnostic RPC framework using Protocol Buffers for serialization. Used for Order-to-Inventory and Order-to-Payment communication.	**Protocol Buffers (Protobuf)** Binary serialization format used by gRPC — faster and smaller than JSON/XML. Service contracts are defined in .proto files.
REST API Representational State Transfer — HTTP-based API style used for client-facing communication between React, Nginx, and primary services.	**Nginx** High-performance open-source web server used as the API Gateway and reverse proxy. Routes incoming HTTP requests to the correct backend service.
Node.js JavaScript runtime built on V8 engine — used for all backend microservices due to its non-blocking I/O model and suitability for event-driven systems.	**MongoDB** Document-oriented NoSQL database. Each microservice maintains its own collection, ensuring data isolation and independent scalability.
Database-per-Service Pattern Each microservice owns its own data store/collection — preventing tight coupling at the data layer. Services cannot query each other's collections directly.	**JWT (JSON Web Token)** Stateless authentication mechanism. The User Service issues JWTs upon login; other services validate the token to authenticate API calls without a central auth check.
RBAC (Role-Based Access Control) Access control mechanism where users are assigned roles (admin, customer), and certain API endpoints are restricted based on the user's role.	**Docker** Containerization platform used to package each microservice along with its dependencies into a portable container.
Docker Compose A tool for defining and running multi-container Docker applications using a single YAML configuration file.	**ActiveMQ** Open-source message broker from Apache implementing JMS (Java Message Service). Used for async communication between the Order Service and Notification Service.
Event-Driven Architecture (EDA) Architecture pattern where services communicate via events. Producers emit events to a broker; consumers react to events independently and asynchronously.	**Message Queue** A durable buffer managed by the broker (ActiveMQ) where messages are stored until the consumer is ready to process them — ensuring reliable delivery.
Circuit Breaker Fault-tolerance design pattern. When a downstream service (e.g., Payment) fails repeatedly, the circuit opens to prevent cascade failures. Implemented using the Opossum library.	**Reverse Proxy** A server (Nginx) that intercepts client requests and forwards them to backend services — hiding internal architecture and enabling SSL termination and load balancing.
Load Balancing Distribution of incoming traffic across multiple service instances for high availability and horizontal scaling — supported by Nginx configuration.	**Bounded Context** A DDD (Domain-Driven Design) concept defining the boundary within which a particular service's domain

	model applies — each microservice represents one bounded context.

5. End-to-End Request Flow

5.1 Phase 1 — Order Placement Flow

6. User logs in via the React frontend. The login request reaches Nginx via REST.
7. Nginx routes /api/auth to User Service, which validates credentials and returns a JWT token.
8. User browses products. Nginx routes /api/products to Product Service (REST).
9. User places an order. Nginx routes /api/orders to Order Service (REST).
10. Order Service calls Inventory Service via gRPC to check/reserve stock.
11. Order Service calls Payment Service via gRPC to process payment.
12. Order is saved in MongoDB Orders collection. Response returned to client.

5.2 Enterprise — Additional Order Event Flow

13. After successful payment, Order Service publishes an 'order.placed' event to ActiveMQ.
14. Notification Service (a consumer) picks up the event from the queue asynchronously.
15. Notification Service sends confirmation notification (email/SMS) to the user.
16. If Payment Service is unavailable, Circuit Breaker opens and a fallback response is returned to the client — preventing service downtime from cascading.

6. GitHub Repository

The complete source code for both Assignment I and Assignment II is available at the following GitHub repository:

https://github.com/milan-sde/MicroServicesProject

The repository contains the following structure:

- assignment1/ — Phase 1 code: User, Product, Order, Inventory, Payment services + Nginx config + React frontend
- assignment2/ — Enterprise code: All Phase 1 services + Notification Service + Docker Compose + ActiveMQ integration + Circuit Breaker implementation
- docs/ — Architecture diagrams and this documentation

7. Conclusion

This project successfully demonstrates the design and implementation of a real-world Smart Order Management System using a modern Microservices Architecture. Starting from a Phase 1 setup with five Node.js services communicating over REST and gRPC, the system was evolved into a production-ready Enterprise Architecture with Docker containerization, ActiveMQ-powered event-driven messaging, JWT-based authentication, role-based access control, and Circuit Breaker fault tolerance.

Key outcomes from both assignments include: practical experience with Nginx as an API Gateway, designing gRPC service contracts with Protocol Buffers, applying the database-per-service pattern with MongoDB, and implementing asynchronous communication using ActiveMQ. The project effectively bridges theoretical microservices principles with hands-on implementation.